



SAP Application Development (S/4 HANA)

Database Access

Motivation

This chapter focuses on the database accesses. You will learn how to use the data dictionary to gain information about data elements, domains and so on. Moreover, you will develop your first ABAP program, which uses database operations.

Prerequisites

Before starting the exercises you should be familiar with SQL and the basic concepts of ABAP programs.

Content

This chapter explains the basic access to the database by using OpenSQL. You will have a look into the hierarchy of data elements in the SAP system. Hence this chapter forms the fundament for developing more complex SAP applications later on.

Lecture notes

All students should be familiar with SQL and the basic concepts of a database as the chapter builds upon this knowledge. Students can go on with their account from previous chapters.

- **Product:** All
- **Level:** Beginner
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Start Eclipse and open your project

Short description: Start Eclipse and open an existing project or create a new one

Start Eclipse and open the ABAP perspective. If an ABAP project exists please open this project. Else create a new ABAP project according to the *Introduction to SAP Programming* chapter. Please refer to your instructor for your username and your password. Ensure having your own package added to the favorites in Eclipse. If you don't have an own package, create a new one according to the mentioned chapter.

Login

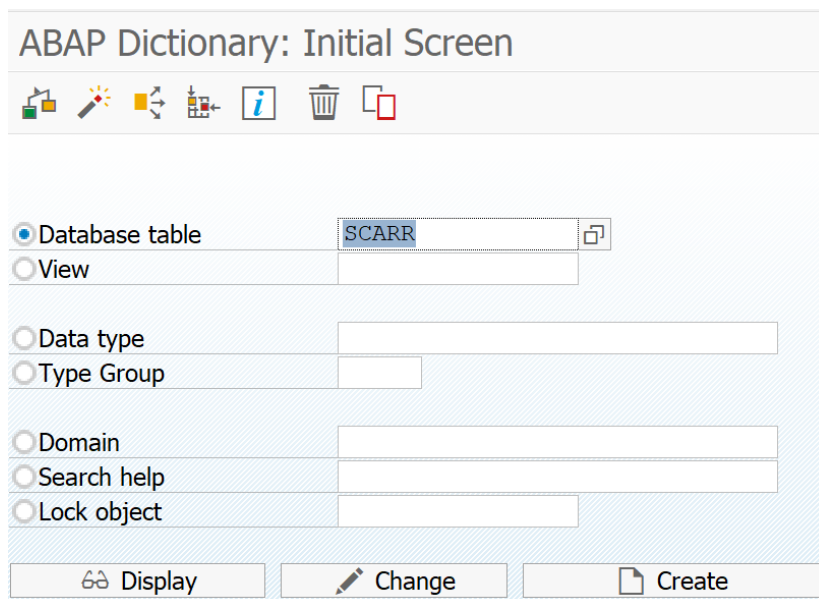
Task 2: First steps in the data dictionary

Short description: Use the data dictionary to explore the structure of table 'SCARR'

Start the internal SAP GUI by clicking the  button or press **Ctrl + 6** on your keyboard. Please start the Data Dictionary now by entering the transaction code **SE11** in the according field.



Choose the option '**Database table**' and type in the table name '**SCARR**'.



Then use the '**Display**' button to have a look at the table definition.

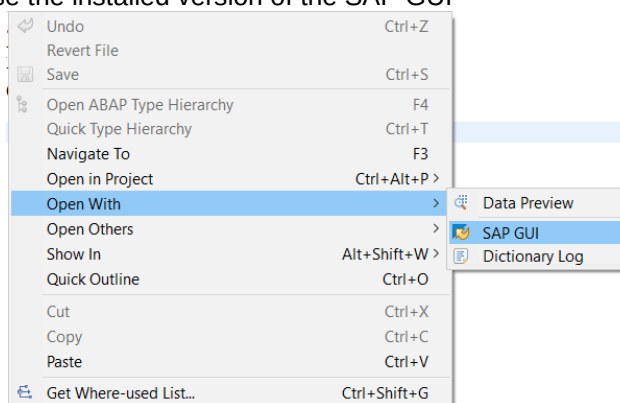
Hint:

If the IDE opens the source code editor instead of the integrated SAP GUI

Hint

```
[D01] SCARR
@EndUserText.label : 'Fluggesellschaft'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_CHARACTER_NUMERIC
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED
define table scarr {
  @AbapCatalog.foreignKey.keyType : #KEY
  @AbapCatalog.foreignKey.screenCheck : true
  key mandt : s_mandt not null
    with foreign key [0..*,1] t000
    where mandt = scarr.mandt;
  key carrid : s_carrid not null;
  carrname : s_carrname not null;
  @AbapCatalog.foreignKey.keyType : #NON_KEY
  @AbapCatalog.foreignKey.screenCheck : true
  currcode : s_currcode not null
    with foreign key [0..*,1] scurx
    where currkey = scarr.currcode;
  url : s_carrurl not null;
}
```

Right-click into the source code to open the **context menu** and select **Open With • SAP GUI** or use the installed version of the SAP GUI



The SAP system brings you directly to the field definition of the table. In this tab you see all the data elements which form the table. Moreover, you see which data element is the key element. Beside the fields tab, there are also several other tabs available:

- **Attributes:** The attributes show you to which package the table is assigned and who did the last changes on which date.
- **Delivery and maintenance:** This is a very important tab as it shows you to which delivery class the table is assigned and if changes to the table are allowed or not.
- **Entry help/check:** This tab shows you if any 'check tables' are used and if there is a 'search help' available.
- **Currency/quantity fields:** The last tab is important when the table contains currency data as it is necessary to choose a reference currency from a customizing table then.

Please switch back to the '**Fields**' tab so you can have a look at the table fields.

Field	Key	Ini...	Data element	Da...	Le...	Deci...	Short Description	Group
MANDT	☒	☒	S_MANDT	CLNT	3	0	Client	
CARRID	☒	☒	S_CARR_ID	CHAR	3	0	Airline Code	
CARRNAME	☐	☒	S_CARRNAME	CHAR	20	0	Airline name	
CURRCODE	☐	☒	S_CURRCODE	CUKY	5	0	Local currency of airline	
URL	☐	☒	S_CARRURL	CHAR	255	0	Airline URL	

You see the fields being defined by table '**SCARR**'. This table is part of the flight data example which SAP includes in every SAP ERP system and contains the IDs and names of the carriers being part of this demo dataset. The table uses field '**MANDT**' and '**CARRID**' as key fields. This is indicated by the Key checkbox. Moreover, you see all the different data elements. For example, the field '**MANDT**' uses the data element '**S_MANDT**' which is linked to the data type '**CLNT**' and has a length of 3.

If you want to explore the data element '**S_MANDT**' now, use the forward navigation of the SAP system and double click on the data element. The data dictionary now jumps directly to the definition of the data element and comes up with the details of the definition.

Forward navigation

☒ Elementary Type
☒ Domain: **MANDT** Client (key field in client-specific.)
 Data Type: **CLNT** Client
 Length: 3
☐ Predefined Type
 Data Type:
 Length: 0
☐ Reference Type
☐ Name of Ref. Type:
☐ Reference to Predefined Type
 Data Type:
 Length: 0

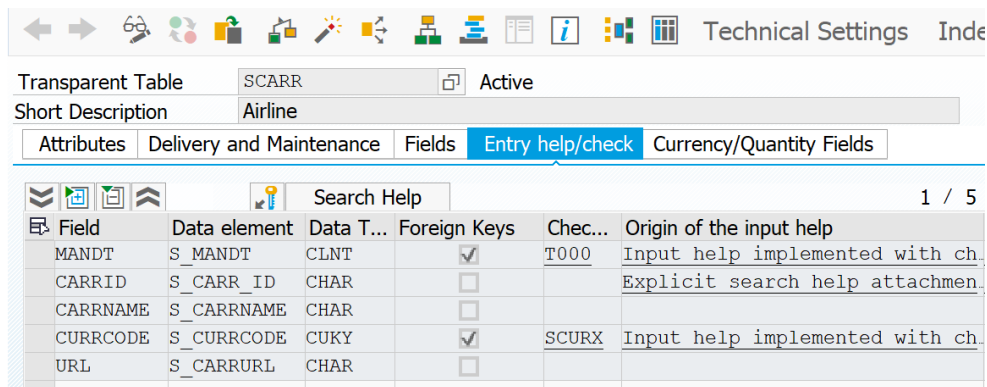
Hint:

Please note that the data element and the table are both in the active state. This is indicated by the '**Active**' text next to the table name and the data elements name. When changing a table or data element in the dictionary the state is changed to '**Inactive**'. If an active version of the table or data element is still in the dictionary, all programs use the active version until the inactive version is activated successfully.

Hint

Please return to the table definition using the '**Back**' button and switch to the tab '**Entry help/check**'. This tab gives you information about implemented entry help and check tables. When working with the table later on, the user may choose an appropriate value to be inserted into the table. To ensure the table integrity the entry help only shows appropriate values. The entry help is built upon the check table. This table only contains appropriate values.

Entry help



Field	Data element	Data T...	Foreign Keys	Chec...	Origin of the input help
MANDT	S_MANDT	CLNT	☒	T000	Input help implemented with ch.
CARRID	S_CARR_ID	CHAR	☐		Explicit search help attachmen.
CARRNAME	S_CARRNAME	CHAR	☐		
CURRCODE	S_CURRCODE	CUKY	☒	SCURX	Input help implemented with ch.
URL	S_CARRURL	CHAR	☐		

In the next step you want to check the foreign key relations. To do so you will consult the graphical representation of the table. The graphical representation can be called by pressing the 'Graphic' button . This button can be found in the toolbar. By pressing the button an additional SAPGui program is started, which is called 'SAP Graphical Editor'. If you do not see a graphic you have to install the 'SAP Graphical Editor' first. To return to SAPGui please use the 'Back' button (F3).

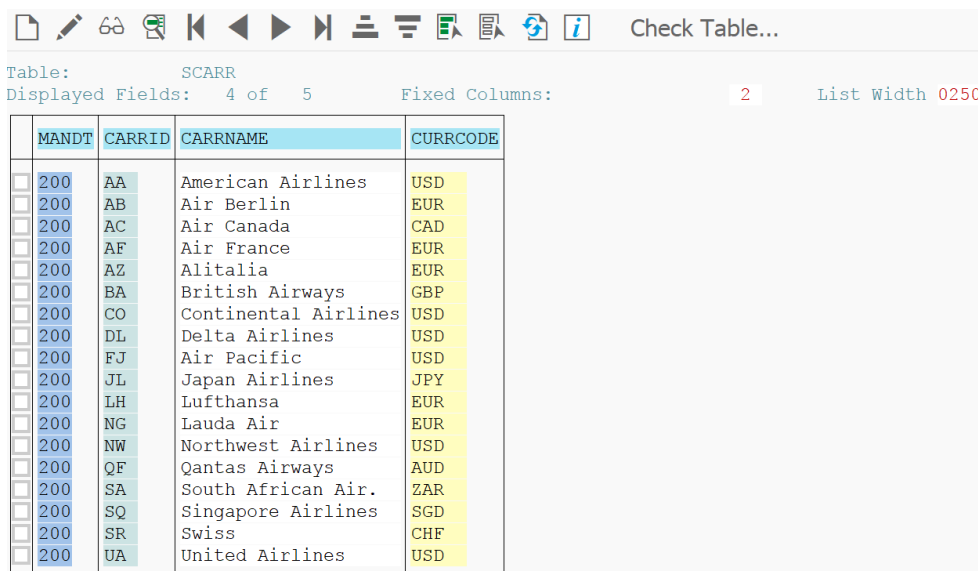
Display graphic

Checking the table content is very important from time to time. You can do this very easily by choosing the following menu path:

(Menu •) Utilities • Table Contents • Display

Menu path

Alternatively, you can press the Contents button . This menu path does not bring you directly to the table content but jumps into the data browser transaction (transaction code **SE16**). All you have to do now is pressing the 'Execute' button (F8). This triggers the SAP system to read all the data from table 'SCARR' and display it. The result should look similar to this:



MANDT	CARRID	CARRNAME	CURRCODE
200	AA	American Airlines	USD
200	AB	Air Berlin	EUR
200	AC	Air Canada	CAD
200	AF	Air France	EUR
200	AZ	Alitalia	EUR
200	BA	British Airways	GBP
200	CO	Continental Airlines	USD
200	DL	Delta Airlines	USD
200	FJ	Air Pacific	USD
200	JL	Japan Airlines	JPY
200	LH	Lufthansa	EUR
200	NG	Lauda Air	EUR
200	NW	Northwest Airlines	USD
200	QF	Qantas Airways	AUD
200	SA	South African Air.	ZAR
200	SQ	Singapore Airlines	SGD
200	SR	Swiss	CHF
200	UA	United Airlines	USD

Task 3: Read and display data from table

Short description: Use SQL statements to read data from a table and display the data using an ABAP program

Before starting programming, have a look at the table 'SPFLI'. The table contains data about flights, which will be displayed in your program later. Please ensure you are familiar with the table definition, especially the primary and foreign keys.

Now close the internal SAP GUI and create a new program called 'Z_&&_###_FLIGHTS' in your existing package Z_\$\$_###. Please use the information provided by your instructor to replace \$\$ and ###. Use your existing transport request or create a new one with an unambiguous description like "Creation of program Z_\$\$_###_FLIGHTS'.

In the first section of your new program you want to define the variables which are needed later on. You need two variables: it_flights as an internal table and wa_flight as a work area. The internal table it_flights will contain the entire content of the database table 'SPFLI' whereas the work area wa_flight only contains one single data set from the internal table.

```
1 REPORT Z_000_FLIGHTS.
2
3 DATA it_flights TYPE TABLE OF spfli.
4 DATA wa_flight TYPE spfli.
```

*Define
variables*

As you can see you define it_flights as a type table of 'SPFLI'. This means the SAP system consults the data dictionary for the structure of the database table SPFLI and creates an internal table with the same structure like SPFLI. The work area wa_flight has the same structure like SPFLI but is not a table, only a single record.

As you want to display the content of the table in your program you have to read all the content from the table. This is done by implementing a simple SQL statement which reads the content from SPFLI into your internal table it_flights

```
6 SELECT * FROM spfli INTO TABLE it_flights.
```

*SQL
statement*

In the last step we want to display the content from it_flights and therefore we loop through the internal table, read the current data record and write it into our work area. The content from the work area will be written to the console.

```
8 LOOP AT it_flights INTO wa_flight.
9   WRITE:/ wa_flight-connid, wa_flight-cityfrom, wa_flight-countryfr,
10    wa_flight-cityto, wa_flight-countryto.
11 ENDLOOP.
```

*Generate
output*

Save and activate your program now. The output should look similar to this:

*Save,
check,
activate*

<i>Database Access</i>			
Database Access			
0017	NEW YORK	US	SAN FRANCISCO
0026	FRANKFURT	DE	NEW YORK
0064	SAN FRANCISCO	US	NEW YORK
0555	ROME	IT	FRANKFURT
0788	ROME	IT	TOKYO
0789	TOKYO	JP	ROME
0790	ROME	IT	OSAKA
0791	OSAKA	JP	ROME
0106	NEW YORK	US	FRANKFURT
1699	NEW YORK	US	SAN FRANCISCO
1984	SAN FRANCISCO	US	NEW YORK
0407	TOKYO	JP	FRANKFURT
0408	FRANKFURT	DE	TOKYO
0400	FRANKFURT	DE	NEW YORK
0401	NEW YORK	US	FRANKFURT
0402	FRANKFURT	DE	NEW YORK
0454	FRANKFURT	DE	SAN FRANCISCO
0455	SAN FRANCISCO	US	FRANKFURT

The program seems to work fine. But how can you know if the SQL statement was executed successfully or not? Well, therefore SAP provides you with the system variable '**sy-subrc**'. You may use the variable to determine if the last action/step in your program was successful or not. You will do this now in your program using an **if-else**-branch. Please use the if-branch to determine if the SQL statement was executed successfully. In this case the program writes all the data to the output whereas when the SQL statement was not executed successfully, the program should return an error message.

```

1  REPORT Z_000_FLIGHTS.
2
3  DATA it_flights TYPE TABLE OF spfli.
4  DATA wa_flight TYPE spfli.
5
6  SELECT * FROM spfli INTO TABLE it_flights.
7
8  IF sy-subrc = 0.
9    LOOP AT it_flights INTO wa_flight.
10     WRITE:/ wa_flight-connid, wa_flight-cityfrom, wa_flight-countryfr,
11            wa_flight-cityto, wa_flight-countryto.
12    ENDLOOP.
13 ELSE.
14   WRITE: 'The SQL Statement was not executed successfully. Please try again later.'.
15 ENDIF.

```

It is necessary to place the if-branch directly after the SQL statement as the variable '**sy-subrc**' only contains the return code of the last instruction. Of course, this is just a very simple error handling but it shows you how to use the system variable to react on failures during runtime. Another one is using exceptions which will be discussed in a later chapter.

Task 4: Use domains, data elements and entry helps

Short description: Use the data dictionary to create an entry help for a new table

Please start the ABAP dictionary by opening the internal SAP GUI and using transaction code SE11.

In the first step you will create a new domain as this is the highest hierarchy level in the ABAP dictionary. Your new domain is named '**Z_\$\$_###_CHAR30**'. Select the '**Domain**' radio button and type in the name into the input field. Then click on the '**Create**' button. The SAP system will bring you directly to the domain details where you have to define a short text and a data type. You can choose a short text on your own. We want to use the data type '**CHAR**' in our domain and the number of characters is limited to '**30**'. If the Output attribute of the Output Characteristics is not available to choose, you may ignore it.

The screenshot shows the 'Domain Details' screen in SAP SE11 for the domain 'Z_000_CHAR30'. The 'Short Description' is 'Domain for character with length of 30'. The 'Definition' tab is active, showing the 'Format' section with 'Data Type' set to 'CHAR' (Character String), 'No. Characters' set to '30', and 'Decimal Places' set to '0'. The 'Output Characteristics' section shows 'Output Length' set to '30', 'Routine' is empty, and the 'Case-sensitive' checkbox is checked.

Now **save and activate** your new domain using your package and your previously used transport request. Return to the ABAP dictionary by pressing the '**Back**' button (F3).

Save and activate

In the next step you want to create a new data type named '**Z_\$\$_###_EN_CHAR30**'. Select the radio button '**Data type**' and type in the name of your new data type.

The screenshot shows the 'Create Type' dialog box for 'Z_EN_CHAR30_000'. It has three radio button options: 'Data element' (which is selected), 'Structure', and 'Table type'. There are green checkmark and red X icons at the bottom right.

The system asks you now if you want to create a Data element or a structure or a table type. Please select the first option '**Data element**'. Maintain the short text and the domain of your new data element. Choose the domain you created in the first step of this task.

The screenshot shows the SAP Data Element Maintenance interface for the data element 'Z_000_EN_CHAR30'. The 'Data Type' tab is selected. Under 'Elementary Type', the 'Domain' radio button is chosen, and the domain 'Z_000_CHAR30' is entered. The 'Data Type' is 'CHAR' and the 'Length' is '30'. Other options like 'Predefined Type', 'Reference Type', and 'Reference to Predefined Type' are visible but not selected.

Switch to the '**Field Label**' tab and maintain the field labels, too. You can define the maximum length of each field label. The maximum length actually used is dependent on the window size.

*Maintain
'Field
Label'*

The screenshot shows the same SAP Data Element Maintenance interface, but now the 'Field Label' tab is selected. It displays a table for defining field labels for the data element 'Z_000_EN_CHAR30'.

	Length	Field Label
Short	10	Title
Medium	20	Title of person
Long	40	Title of person
Heading	40	Title of person

Save and activate your new data element using your transport request and package.

*Save and
activate*

The next step is the creation of your database table. The table will contain some exemplary titles of persons and will therefore function as a check table. The name of the new table is '**Z\$\$\$#_TITLE**'. Please note that because of naming conventions there is no underscore after Z. Create it analogue to the creation of the Domain and Data type. Maintain the short text and then choose '**Application table (master and transaction data)**' as the '**Delivery Class**' and '**Display/Maintenance Allowed**' as the value for '**Data Browser/Table View Maint.**'.

Create table

Dictionary: Change Table

Transparent Table: Z000_TITLE New(Revised)

Short Description: Title Table

Attributes | **Delivery and Maintenance** | Fields | Entry help/check | Currency/Quantity Fields

Delivery Class: A Application table (master and transaction data)

Data Browser/Table View Maint.: Display/Maintenance Allowed

Switch to tab **'Fields'** to maintain the fields of your table and define the first field **'Mandt'** with data element **MANDT** and select the **'Key'** and **'Initial value'** checkbox for it. This is necessary for client dependent tables. Afterwards define the second field **'Title'**. The title field is also used as the primary key. So select the checkbox **'Key'** and **'Initial Value'**. Furthermore you have to define the corresponding data element. Here you choose the created data element **'Z_\$\$_###_EN_CHAR30'**. Please use the F4 help to avoid any typing failures and save all your changes.

Tab Fields

Transparent Table: Z000_TITLE Active

Short Description: Title Table

Attributes | Delivery and Maintenance | **Fields** | Entry help/check | Currency/Quantity Fields

Field	Key	Ini...	Data element	Data Type	Le...	D...	Short Description
MANDT	☒	☒	MANDT	NT	3	0	Client
TITLE	☒	☒	Z_000_EN_CHAR30	CHAR	30	0	Data element for domain Z_000_CHAR30

Now switch to the technical settings of your table by using the button:

Technical Settings

Technical settings

The technical settings describe how the table will be stored later on and if data records of the table are buffered or not. The maintenance of the data class is mandatory: please choose **'APPL0'** and the size category **'0'**.

Revised<->Active

Name: Z000_TITLE Transparent Table

Short Descript.: Title Table

Last Changed: DEV-001 26.08.2016

Status: Actv. Saved

General Properties | DB-Specific Properties

Logical Storage Parameters

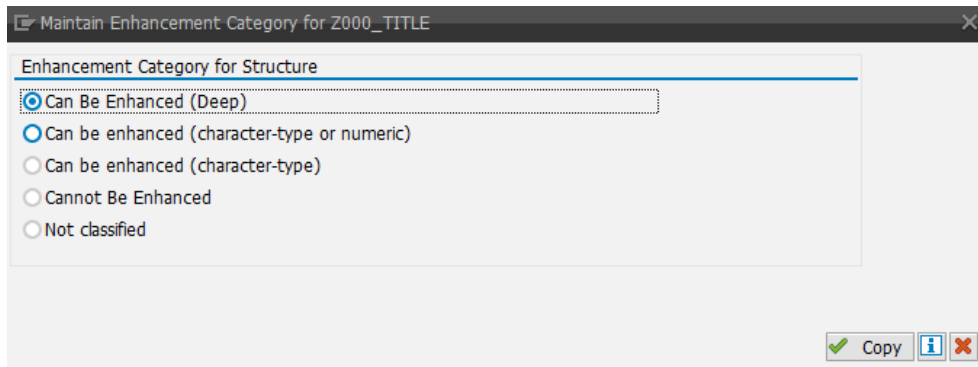
Data Class: APPL0 Master Data, Transparent Tables

Size Category: 0 Exected data records 0 to 9.300

Leave the rest of the settings as they are and save the technical settings. After saving the settings, go back (F3) to the table maintenance. Now define an enhancement category by using following menu path:

(Menu •) Extras • Enhancement Category

Confirm the message and select the first option.



Activate your table now and ignore all issued warnings.

In the next step you will enter some data records into the table. This can be done easily using the '**Data browser**'. The Data browser can be accessed via the menu path:

Menu • Utilities • Table Contents • Create Entries

Menu path

In case the menu option is displayed in gray (can not be selected), make sure you did not forget to change the '**Data Browser/Table View Editing**' in the **Delivery and Maintenance** tab to '**X Display/Maintenance**' Allowed as described above.

The SAP system jumps directly to a new program where you can add new data records into the table. To add a new entry into the table, type in the title into the input field and then press the '**Save**' button.

Insert data record

Add the following titles: PhD., Dr., Prof. Dr., Prof. Dr. h.c. mult. After you entered all titles please go back to the data dictionary.

In the next step we want to create another table named '**Z\$\$\$## PERSON**'. The table will contain data about persons. You will integrate the previous generated table '**Z\$\$\$##_TITLE**' into the new table by using a foreign key. Create the table using the following fields: PERSONID, NAME, FAMILY_NAME and TITLE. Use also **Delivery Class A** and **Allow Display/Maintenance** for this table. Refer to the following figure:

<

Hint:

When defining table fields you may either use predefined data types or data elements. The best way to define your table fields is to define all fields using predefined data types at first and then define table fields using data elements. Use the button **Data Element** / **Built-In Type** to switch the modus.

Hint

Now you want to define the foreign key connection between both tables whereas the 'TITLE' field should be linked to the table 'Z\$\$\$_TITLE'. Therefore select the field

'TITLE' and press the 'Foreign Key' button  in the toolbar. The SAP system comes up with a pop-up and asks you for the check table. Please select your title table 'Z\$\$\$_TITLE' and press 'Enter'. The system will automatically read the repository information and will propose a foreign key definition using the field names from both tables.

Change Foreign Key Z000_PERSON-TITLE

Short text: Foreign key title
Check table: Z000_TITLE Generate proposal

Check table	ChkTabFld	For.key table	Foreign Key Field	Generic	Constant
Z000_TITLE	MANDT	Z000_PERSON	MANDT	☐	
Z000_TITLE	TITLE	Z000_PERSON	TITLE	☐	

Screen check
☒ Check required Error message MsgNo AArea

Semantic attributes
Foreign key field type: ☒ Not Specified
☐ Non-key-fields/candidates
☐ Key fields/candidates
☐ Key fields of a text table
Cardinality: :

Copy

Now finally **define an enhancement category as deep** (see page 10 and 11) and specify the **Data Class APPL0** and **Size Category 0** in the **technical settings**. Please **save** and **activate your new table** after you maintained the technical settings. Prompted warnings can be ignored. To prove if the check table was defined successfully you will now create a program. Leave the data dictionary.

Close the integrated SAP GUI and create a new program called 'Z_\$\$_###_PERSON'. Use the following code fragment to define parameters in your program:

```

1 REPORT Z_000_PERSON.
2
3 parameters: pa_title type z000_person-title,
4 pa_name type z000_person-name,
5 pa_fam type z000_person-family_name.

```

In case the IDE marks **z\$\$\$##_person** as an unknown data type you probably forgot to save and activate the database table.

**Save, check,
activate, test**

Save, check and **activate** your new program. Now when testing your new program the SAP system comes up with an entry help on the '**PA_TITLE**' input field. This is because of the foreign key definition. All the values come from the title table.

Challenge: Table joins and data aggregation

Task: Your task is to join the tables SFLIGHT and SBOOK via an inner join in order to sum up the information of the field luggweight (SBOOK-luggweight) per plane type (SFLIGHT-planetype).

1. Therefore firstly **create** a new **program** called **Z_\$\$_###_LUGGWEIGHT**
2. **Declare** an **own** local **datatype** structure, called '**FLIGHTWEIGHT**' consisting of a **field** to store SUM information called **weight_sum** of type "*Packed Number*" and a field to store the plane type called **planetype** (use the TYPES statement to declare own datatypes)
3. **Declare** an internal table called **it_flight** and a workarea called **wa_flight** that are based on the newly created datatype.
4. Write an **SQL** command to read the **SUM of FLIGHTWEIGHT grouped by planetypes** into your **it_flight** table. Use the internet to come up with a correct SQL syntax that combine the inner table joins with the SUM aggregate. You may find the following websites helpful:

https://help.sap.com/doc/abapdocu_750_index_htm/7.50/de-DE/abapselect_join.htm

(Use the field CONNID for the table join)

https://help.sap.com/doc/abapdocu_750_index_htm/7.50/en-US/abapselect_aggregate.htm

5. **Loop** through your **internal table** and **print** the **output** to the screen

Your solution screens should look like this:

Display the luggage weight

747-400	3.030.171,20
767-200	2.103.188,30
A319-100	210.353,00
A320-200	515.059,50
A340-600	3.125.308,40
A380-800	9.146.519,60